

Cours MVSI

Modélisation et Vérification des Systèmes Informatiques

Modélisation, spécification et vérification (II)

Dominique Méry
Telecom Nancy, Université de Lorraine
(8 octobre 2026 at 6:31 P.M.)

① Le langage PlusCal

Checking a program or an algorithm using TLA⁺ and PlusCal

Annotation of PlusCal programs

Define algorithms and processes in PlusCal

Macros et procédures

② Summation of the n first integers

③ Principe(s) d'induction

④ Méthode de preuves de propriétés d'invariance

① Le langage PlusCal

Checking a program or an algorithm using TLA⁺ and PlusCal

Annotation of PlusCal programs

Define algorithms and processes in PlusCal

Macros et procédures

② Summation of the n first integers

③ Principe(s) d'induction

④ Méthode de preuves de propriétés d'invariance

① Le langage PlusCal

Checking a program or an algorithm using TLA⁺ and PlusCal

Annotation of PlusCal programs

Define algorithms and processes in PlusCal

Macros et procédures

② Summation of the n first integers

③ Principe(s) d'induction

④ Méthode de preuves de propriétés d'invariance

- ▶ Definition of a simple algorithmic language.
- ▶ Specific comments within the text of the TLA⁺ module, between (*) and *)
- ▶ Definition of an algorithm as a sequence of definitions of sequential processes that may share variables :

`--algorithm name { definitions }`

- ▶ Generation of a TLA⁺ specification with the introduction of a new variable 'pc' modelling control.
- ▶ Introduction of a variable 'pc' to model the control of the algorithm.
- ▶ The ToolBox tool has a translation feature.

Computing the quotient and the remainder of two naturals

```
1  /* data structure */
2  struct result {
3      int quotient;
4      int reste;
5  };
6  /* Function for euclidian division */
7  struct result division(int x, int y)
8  {
9      struct result r;
10     r.quotient = 0;
11     r.reste = x;
12
13     while (r.reste >= y)
14     {
15         r.reste = r.reste - y;
16         r.quotient = r.quotient + 1;
17     }
18     return r;
19 }
```

- ▶ Translating the function called *division* as an algorithmic fragment.

Listing 1 – Module TLA+

```
1  ----- MODULE division0 -----
2  EXTENDS TLC, Integers, Naturals
3  CONSTANTS x0,y0,M,N
4  -----
5  ASSUME x0\in 1..M /\ y0 \in 1..N
6  -----
7  (*
8  --algorithm division {
9  variables x = x0,y=y0,quotient = 0, reste = x;
10 {
11     while (reste >= y)
12     {
13         reste := reste - y;
14         quotient := quotient + 1;
15     }
16 }}
17 *)
18
19 =====
```

- ▶ Assuming that $x0$ and $y0$ are in two intervals defined the two constants M and N .
- ▶ Preconditions, Postconditions and Safety Properties are defined as follow :

Listing 2 – Module TLA+

```
1 pre == x0 \in Nat /\ y0 \in Nat /\ y0#0
2 post ==
3     /\ pre /\ x=x0 /\ y=y0
4     /\ x0 = y0 * quotient + reste
5     /\ 0 \leq reste /\ reste < y0
6 D(X) == X \in -M..M
7 safetypc == pc="Done" => post
8 safetyrte == D(x) /\ D(y) /\ D(quotient) /\ D(reste)
9 safecontrol == pc \in {"Lbl_1", "Done"}
10 safety == safetypc /\ safetyrte /\ safecontrol
```

- ▶ Using the translation PCAL of the toolbox produces the following TLA⁺ module.

- Using the translation PCAL of the toolbox produces the following TLA⁺ module.

Listing 4 – Module TLA+

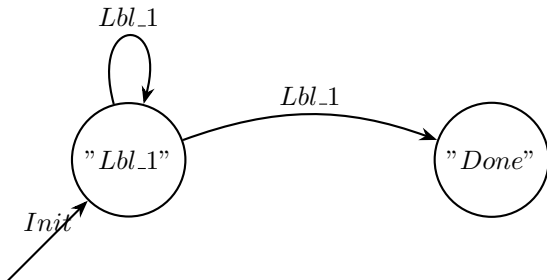
```
1
2 Lbl_1 == /\ pc = "Lbl_1"
3           /\ IF reste >= y
4              THEN /\ reste' = reste - y
5                   /\ quotient' = quotient + 1
6                   /\ pc' = "Lbl_1"
7              ELSE /\ pc' = "Done"
8                   /\ UNCHANGED << quotient, reste >>
9           /\ UNCHANGED << x, y >>
10
11 (* Allow infinite stuttering to prevent deadlock on termination. *)
```

- Using the translation PCAL of the toolbox produces the following TLA⁺ module.

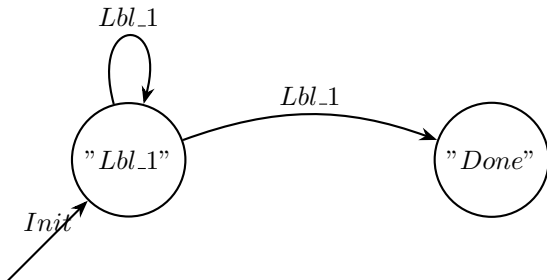
Listing 5 – Module TLA+

```
1  (* Allow infinite stuttering to prevent deadlock on termination. *)
2  Terminating == pc = "Done" /\ UNCHANGED vars
3
4  Next == Lbl_1
5          \/ Terminating
6
7  Spec == Init /\ [][Next]_vars
8
9  Termination == <>(pc = "Done")
10
11 \* END TRANSLATION
```

- ▶ *Lbl_1* is a label assigned to a while loop.
- ▶ The translation process adds a new label called *Done* and actions are called using the labels



- ▶ *Lbl_1* is a label assigned to a while loop.
- ▶ The translation process adds a new label called *Done* and actions are called using the labels



Listing 7 – Module TLA+

```
1 safecontrol == pc \in {"Lbl_1", "Done"}
2 safety == safetypc /\ safetyrtc /\ safecontrol
```

Listing 8 – Module TLA+

```

1  ----- MODULE label1 -----
2  EXTENDS TLC, Integers, Naturals
3  CONSTANTS x0,y0
4  -----
5  (*
6  --algorithm ex {
7  variables x = x0,y=y0;
8  {
9      while (x # 0)
10     {
11         while (y # 0)
12         {
13             y:=y-1;
14         };
15         x:=x-1;
16     }
17 }
18 }
19
20 *)

```

Listing 9 – Module TLA+

```
1  \* BEGIN TRANSLATION (chksum(pcal) = "3ad78380" /\ chksum(tla) = "  
    a33d9efe")  
2  VARIABLES x, y, pc  
3  
4  vars == << x, y, pc >>  
5  
6  Init == (* Global variables *)  
7          /\ x = x0  
8          /\ y = y0  
9          /\ pc = "Lb1_1"
```

Listing 10 – Module TLA+

```
1 Lbl_1 == /\ pc = "Lbl_1"  
2          /\ IF x # 0  
3              THEN /\ pc' = "Lbl_2"  
4                  ELSE /\ pc' = "Done"  
5          /\ UNCHANGED << x, y >>  
6
```

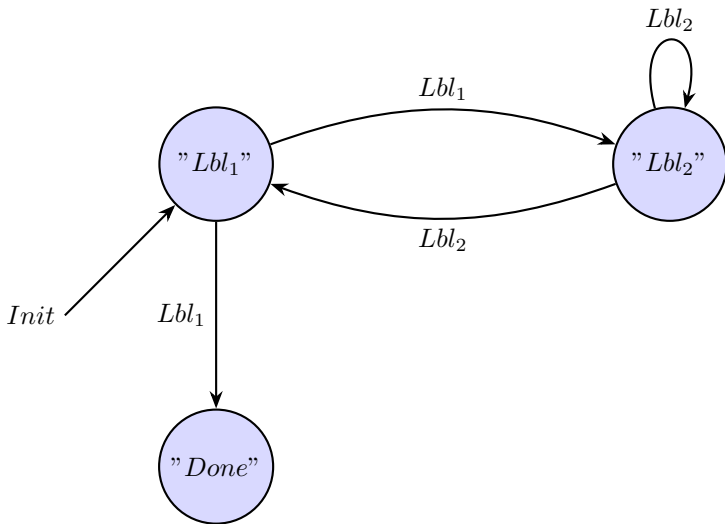
Listing 11 – Module TLA+

```
1 Lbl_2 == /\ pc = "Lbl_2"
2          /\ IF y # 0
3             THEN /\ y' = y-1
4                  /\ pc' = "Lbl_2"
5                  /\ x' = x
6             ELSE /\ x' = x-1
7                  /\ pc' = "Lbl_1"
8                  /\ y' = y
9
```

Listing 12 – Module TLA+

```
1
2 (* Allow infinite stuttering to prevent deadlock on termination. *)
3 Terminating == pc = "Done" /\ UNCHANGED vars
4
5 Next == Lbl_1 \/ Lbl_2
6         \/ Terminating
7
8 Spec == Init /\ [][Next]_vars
9
10 Termination == <>(pc = "Done")
11
12 \* END TRANSLATION
```

- ▶ The set of labels is $\{ "Lbl_1", "Lbl_2", "Done" \}$.
- ▶ Actions are defined as atomic computations between the three labels and each label defines an action from the label to others possible labels.
- ▶ Take the example of *Lbl_2* either looping or getting out to *Lbl_1*.



① Le langage PlusCal

Checking a program or an algorithm using TLA⁺ and PlusCal

Annotation of PlusCal programs

Define algorithms and processes in PlusCal

Macros et procédures

② Summation of the n first integers

③ Principe(s) d'induction

④ Méthode de preuves de propriétés d'invariance

- ▶ Each statement `stmt` can be labelled by a label `lab` and the resulting labelled statement is `lab:stmt`.
- ▶ The rule of labelling is to label at least the while statements and possibly to add other observation points to express the control flow.
- ▶ Two syntaxes are possible for PlusCal : P-syntax (prolix) and C-syntax (compact).

Listing 13 – Module TLA+

```

1  ---- MODULE label2----
2  EXTENDS Naturals
3  (* --algorithm Exemple
4  variables
5      i = 0, j = 0, x = 0;
6  begin
7  L1: while i < 3 do
8      L2:      j := 0;
9      L3:      while j < 3 do
10         L4:      if i < 2 then
11         L5:      if j < 2 then
12             x := x + 1;
13         else
14             x := x + 2;
15         end if;
16     else
17 L6:      if j = 0 then
18         x := x + 3;
19     else
20         x := x + 4;
21     end if;
22     end if;
23 L7:      j := j + 1;
24     end while;
25 L8:      i := i + 1;
26     end while;
27 L9:      skip;

```

```

----- MODULE exemple1 -----
EXTENDS Naturals, Integers, TLC
CONSTANTS x0,y0,z0,min,max,undef

-----

(* precondition *)
ASSUME x0 = y0 + 3*z0

-----

(*
--algorithm exemple1 {
    variables x=x0,
               y = y0,
               z=z0;

{
10: assert x = y + 3*z /\ /\ y=y0 /\ z=z0 ;
    x := y+3*z;
11: assert x = y0+3*z0 /\ /\ y=y0 /\ z=z0 ;
}
}

```

Exemple (II)

```
----- MODULE exemple1 -----  
.....  
  
-----  
ISDEF(X,Y) == X # undef => X \in Y  
DD(X) == X # undef => X \in min..max  
-----  
  
i ==  
  /\ pc \in {"l0","l1","Done"}  
  /\ ISDEF(x,Int) /\ ISDEF(y,Int) /\ ISDEF(z,Int)  
  /\ pc = "l0" => x = y + 3*z  
  /\ pc = "l1" => x+y+z \geq y  
post ==      x = y0+3*z0 /\ y=y0 /\ z=z0  
  
safetyrte == DD(x) /\ DD(y) /\ DD(z)  
safetypc == pc="Done" => post  
=====
```

① Le langage PlusCal

Checking a program or an algorithm using TLA⁺ and PlusCal

Annotation of PlusCal programs

Define algorithms and processes in PlusCal

Macros et procédures

② Summation of the n first integers

③ Principe(s) d'induction

④ Méthode de preuves de propriétés d'invariance

Listing 14 – Module TLA+

```
1  ---- MODULE module_name ----
2  \* TLA+ code
3
4  (* --algorithm algorithm_name
5  variables global_variables
6
7  process p_name = ident
8  variables local_variables
9  begin
10     \* pluscal code
11 end process
12
13 process p_group \in set
14 variables local_variables
15 begin
16     \* pluscal code
17 end process
18
19 end algorithm; *)
20 =====
```

- ▶ Global variables are declared at the level of the algorithm and are shared by all processes, which may read and write to them in accordance with the atomicity rule.
- ▶ Local variables are defined at the level of each process, which is a sequential process.
- ▶ Parallelism is achieved through non-deterministic interleaving of the actions of the different processes.
- ▶ Control is introduced by the variable `pc`, which is an array of size equal to the number of possible concurrent processes.

① Le langage PlusCal

Checking a program or an algorithm using TLA⁺ and PlusCal

Annotation of PlusCal programs

Define algorithms and processes in PlusCal

Macros et procédures

② Summation of the n first integers

③ Principe(s) d'induction

④ Méthode de preuves de propriétés d'invariance

```
1 macro Name(var1, ...)
2 begin
3   \* something to write
4 end macro;
```

```
1 procedure Name(arg1, ...)
2 variables var1 = ... \* not \in, only =
3 begin
4   Label:
5   \* something
6   return;
7 end procedure;
```

① Le langage PlusCal

Checking a program or an algorithm using TLA^+ and PlusCal

Annotation of PlusCal programs

Define algorithms and processes in PlusCal

Macros et procédures

② Summation of the n first integers

③ Principe(s) d'induction

④ Méthode de preuves de propriétés d'invariance

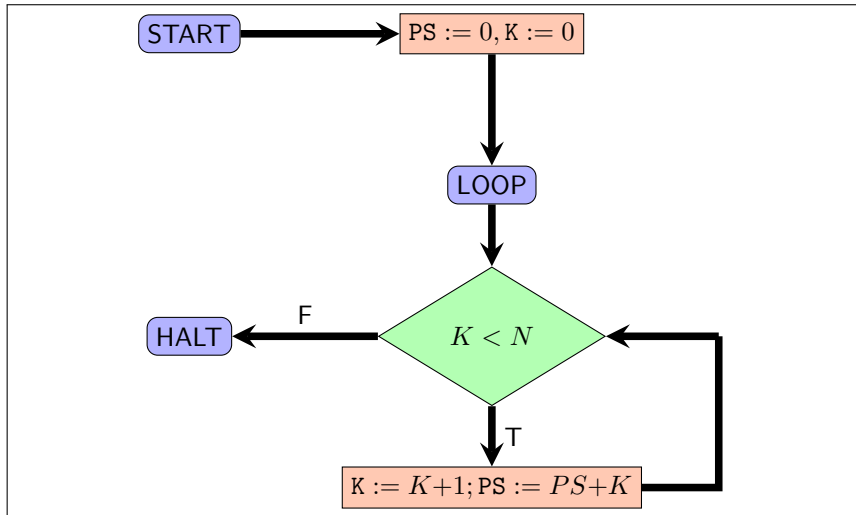
- ▶ Un programme ou un algorithme peuvent être annotés ou commentés.
- ▶ Un commentaire est une information pertinente destinée à être vue ou lue et qui a une importance relative dans l'esprit du concepteur.
- ▶ Un commentaire indique une information sur les données, sur les variables et donc sur l'état supposé du programme à l'exécution.
- ▶ Un commentaire est une annotation du texte du code qui nous permet de communiquer une information sémantique :
 - *à ce point, la variable k est plus petite sur n*
 - *l'indice e fait référence à une adresse licite de t et cette valeur est toujours positive*
 - *la somme des variables est positive*
- ▶ Les annotations peuvent être systématisées et obéir à une syntaxe spécifique définissant le langage d'annotations ;

```
1 /*@ assert l1: z >= 3 && y == 3; */  
2 z = z + y;  
3 /*@ assert l2: z >= 6 && y == 3; */
```

Calculer la somme des n premiers entiers (v0)

```
1  int fS(int n) {  
2      int ps = 0;  
3      int k = 0;  
4      while (k < n) {  
5          k = k + 1;  
6          ps = ps + k;  
7      };  
8      return ps;  
9  }
```

Calculer la somme des n premiers entiers (flowchart)



Calculer la somme des n premiers entiers (v0)

```
1 // pre n>=0;
2 // post ps == n*(n+1) / 2;
3
4 int fS(int n) {
5     int ps = 0;
6     int k = 0;
7     while (k < n) {
8         k = k + 1;
9         ps = ps + k;
10    };
11    return ps;
12 }
13
14
15 int main()
16 {
17
18 }
```

Calculer la somme des n premiers entiers (v0)

```
1 // pre n>=0;
2 // post ps == n*(n+1) / 2;
3
4 int fS(int n) {
5     int ps = 0;
6     int k = 0;
7     while (k < n) {
8         // ps = k*(k+1) / 2;
9         k = k + 1;
10        ps = ps + k;
11    };
12    // ps = n*(n+1) / 2;
13    return ps;
14 }
15
16
17 int main()
18 {
19
20 }
```


Definition of S(n) and IS(n)

$$\forall n \in \mathbb{N} : S(n) = \sum_{k=0}^n k$$

$$\begin{cases} IS(0) = 0 \\ n \geq 0, IS(n+1) = IS(n) + (n+1) \end{cases}$$

Property for S(n) and IS(n)

$$\forall n \in \mathbb{N} : S(n) = IS(n)$$

- ▶ base 0 : $S(0) = 0 = IS(0)$
- ▶ induction $i+1$: $\forall j \in 0..i : S(j) = IS(j)$
 - $S(i+1) = \sum_{k=0}^{i+1} k$ (definition)
 - $S(i+1) = (\sum_{k=0}^i k) + i+1$ (property of summation)
 - $S(i+1) = S(i) + (i+1)$ (by definition of S(i))
 - $S(i+1) = IS(i) + (i+1)$ (by inductive assumption on S(i) et IS(j))
 - $S(i+1) = IS(i) + (i+1) = IS(i+1)$ (by definition of IS(i+1))
- ▶ conclusion : $\forall i \in \mathbb{N} : S(i) = IS(i)$ (by induction principle)

- ▶ $\forall n \in \mathbb{N} : S(n) = \sum_{k=0}^n k$
- ▶ $\left[\begin{array}{l} IS(0) = 0 \\ n \geq 0, IS(n+1) = IS(n) + n \end{array} \right.$
- ▶ $\forall n \in \mathbb{N} : S(n) = IS(n)$
- ▶
 - base 0 : $S(0) = 0$
 - induction $k+1$: $S(k+1) = S(k) + k + 1$
 - step $k+1$: $S(k+1) = S(k) + k + 1$
- ▶ $S(k) = ps$: current value of ps is $S(k)$
- ▶ $S(k-1) = ops$: current value of ops is $S(k-1)$
- ▶ step $k+1$: $ps = ops + k + 1$

Calculer la somme des n premiers entiers (v2)

```
1
2 int fS(int n) {
3     int ps = 0;
4     int k = 0;
5     int ok=k, ops = 0;
6     while (k < n) {
7         /*@ assert 0 <= k && k <= n
8            && ps == S(k) && ops == S(ok);    */
9            ok=k;ops=ps;
10            k = ok + 1;
11            ps = ops + k;
12        /*@ assert 0 <= k && k <= n && ps == S(k)
13            && ops == S(ok);    */
14    };
15    return ps;
16 }
```

Calculer la somme des n premiers entiers

```
1  /*@ axiomatic S {
2    @ logic integer S(integer n);
3    @ axiom S_0: S(0) == 0;
4    @ axiom S_i: \forall integer i; i > 0 ==> S(i) == S(i-1)+i;
5    @ } */
6
7
8  /*@ requires n >= 0;
9    assigns \nothing ;
10   ensures \result == S(n);
11  */
12  int fS(int n) {
13    int ps = 0;
14    int k = 0;
15    int ok=k,ops=ps;
16    /*@ loop invariant 0 <= k && k <= n && ps == S(k) && ops == S(ok)
17      ;
18      loop assigns ps, k,ops,ok;
19    */
20    while (k < n) {
21      /*@ assert l0: 0 <= k && k <= n && ps == S(k) && ops == S(ok);    */
22      ops=ps;ok=k;
23      k = ok + 1;
24      ps = ops + k;
25      /*@ assert l1: 0 <= k && k <= n && ps == S(k) && ops == S(ok);
26        */
27    };
28    /*@ assert ps == S(n);    */
29    return ps;
```

- ▶ Définition des fonctions mathématiques nécessaires pour exprimer le calcul de la somme des n premiers nombres entiers.
- ▶ Expression des résultats intermédiaires appelés *sommes partielles*
- ▶ Relation entre la preuve par induction et la forme du corps de l'itération.
- ▶ Induction et calcul sont liés.

- ▶ Définition des fonctions mathématiques nécessaires pour exprimer le calcul de la somme des n premiers nombres entiers.
- ▶ Expression des résultats intermédiaires appelés *sommes partielles*
- ▶ Relation entre la preuve par induction et la forme du corps de l'itération.
- ▶ Induction et calcul sont liés.

$$x_0 \xrightarrow{P} x \quad (1)$$

$$x_0 \xrightarrow{\star} x \quad (2)$$

$$x_0 \longrightarrow x_1 \longrightarrow \dots \longrightarrow x \quad (3)$$

$$x_0 \longrightarrow x_1 \longrightarrow \dots \longrightarrow x_n \xrightarrow{step} x \quad (4)$$

- ① Le langage PlusCal
 - Checking a program or an algorithm using TLA⁺ and PlusCal
 - Annotation of PlusCal programs
 - Define algorithms and processes in PlusCal
 - Macros et procédures
- ② Summation of the n first integers
- ③ Principe(s) d'induction
- ④ Méthode de preuves de propriétés d'invariance



On convient des notations suivantes équivalentes :
 $x \in E$ est équivalent à $E(x)$ pour toute valeur $x \in \mathbf{Vals}$.
Cette simplification permet de relier un ensemble $U \subseteq \mathbf{Vals}$ à une assertion $U(x)$ en considérant que $U(x)$ et $x \in U$ désigne le même concept.

Les deux expressions suivantes sont équivalentes :

- ▶ $\forall x_0, x \in \mathbf{VALS}. \text{Init}(x_0) \wedge \text{NEXT}^*(x_0, x) \Rightarrow A(x)$
- ▶ $\forall x \in \mathbf{VALS}. (\exists x_0. x_0 \in \mathbf{VALS} \wedge \text{Init}(x_0) \wedge \text{NEXT}^*(x_0, x)) \Rightarrow A(x)$

i

- ▶ $\forall x_0, x \in \mathbf{VALS}. \text{Init}(x_0) \wedge \text{NEXT}^*(x_0, x) \Rightarrow A(x)$
- ▶ $\forall x \in \mathbf{VALS}. (\exists x_0. x_0 \in \mathbf{VALS} \wedge \text{Init}(x_0) \wedge \text{NEXT}^*(x_0, x)) \Rightarrow A(x).$
- ▶ $\text{REACHABLE}(M) = \{u | u \in \mathbf{VALS} \wedge (\exists x_0. x_0 \in \mathbf{VALS} \wedge \text{Init}(x_0) \wedge \text{NEXT}^*(x_0, u))\}$ est l'ensemble des états accessibles à partir des états initiaux et on doit montrer la propriété de sûreté $A(x)$ en montrant l'inclusion des ensembles (model-checking) :

$$\text{REACHABLE}(M) \subseteq \{u | u \in \mathbf{VALS} \wedge A(u)\}$$

Soit $(Th(s, c), x, \text{VALS}, \text{Init}(x), \{r_0, \dots, r_n\})$ un modèle relationnel M d'un système S.

Une propriété $A(x)$ est une propriété de sûreté pour le système S, si et seulement s'il existe une propriété d'état $I(x)$, telle que :

$$\forall x, x' \in \text{VALS} : \begin{cases} (1) \text{Init}(x) \Rightarrow I(x) \\ (2) I(x) \Rightarrow A(x) \\ (3) I(x) \wedge \text{NEXT}(x, x') \Rightarrow I(x') \end{cases}$$

La propriété $I(x)$ est appelée un invariant inductif de S et est une propriété de sûreté particulière plus forte que les autres propriétés de sûreté.

Soit une propriété $I(x)$ telle que :

$$\forall x, x' \in \text{VALS} : \begin{cases} (1) \text{ Init}(x) \Rightarrow I(x) \\ (2) I(x) \Rightarrow A(x) \\ (3) I(x) \wedge \text{NEXT}(x, x') \Rightarrow I(x') \end{cases}$$

Alors $A(x)$ est une propriété de sûreté pour le système S modélisé par M .

Soient x et $x' \in \text{VALS}$ tels que $\text{INIT}(x) \wedge \text{NEXT}^*(x, x')$.

- ▶ On peut construire une suite telle que :

$$(x = x_0) \xrightarrow{\text{NEXT}} x_1 \xrightarrow{\text{NEXT}} x_2 \xrightarrow{\text{NEXT}} \dots \xrightarrow{\text{NEXT}} (x_i = x').$$

- ▶ L'hypothèse (1) nous permet de déduire $I(x_0)$.
- ▶ L'hypothèse (3) nous permet de déduire $I(x_1), I(x_2), I(x_3), \dots, I(x_i)$. En utilisant l'hypothèse (2) pour x' , nous en déduisons que x' satisfait A .

$$\forall x_0, x \cdot x, y \in \text{VALS} \wedge \text{Init}(x_0) \wedge x_0 \xrightarrow[\text{Next}]{\star} x \Rightarrow A(x)$$

PROUVONS QUE : il existe une propriété $I(x)$ telle que :

$$\forall x, x' \in \text{VALS} : \begin{cases} (1) \text{Init}(x) \Rightarrow I(x) \\ (2) I(x) \Rightarrow A(x) \\ (3) I(x) \wedge \text{NEXT}(x, x') \Rightarrow I(x') \end{cases}$$

- Nous considérons la propriété suivante :

$$I(x) \hat{=} \exists x_0 \in \text{VALS} \cdot \text{Init}(x_0) \wedge x_0 \xrightarrow[\text{Next}]{\star} x.$$

- $I(x)$ exprime que la valeur x est accessible à partir d'une valeur initiale x_0 .
- Les trois propriétés sont simples à vérifier pour $I(x)$. $I(x)$ est appelé le plus fort invariant de l'algorithme $\mathcal{A}$.

- ▶ P. et R. Cousot développent une étude complète des propriétés d'invariance et de sûreté en mettant en évidence correspondances entre les différentes méthodes ou systèmes proposées par Turing, Floyd, Hoare, Wegbreit, Manna ... et reformulent les principes d'induction utilisés pour définir ces méthodes de preuve (voir les deux cubes des 16 principes).
- ▶ Deux types de principes sont proposés : assertionnel et relationnel.
- ▶ Nous utilisons l'expression de propriété de sûreté, alors que généralement il s'agit d'une propriété d'invariance ($\square$ propriété) et d'invariant au lieu d'invariant inductif.

Complétude et correction

$$\forall x_0, x \in \text{VALS}. \text{Init}(x_0) \wedge \text{NEXT}^*(x_0, x) \Rightarrow A(x).$$

si, et seulement si,

il existe $I \in \mathcal{P}(\text{VALS})$

$$\forall x_0, x, x' \in \text{VALS} : \begin{cases} (1) \text{Init}(x_0) \Rightarrow I(x_0) \\ (2) I(x) \Rightarrow A(x) \\ (3) I(x) \wedge \text{NEXT}(x, x') \Rightarrow I(x') \end{cases}$$

si, et seulement si,

$$\exists i \in \mathcal{P}(\text{VALS}). \begin{cases} (1) \text{Init} \subseteq i \\ (2) i \subseteq A \\ (3) \forall x, x' \in \text{VALS}. i(x) \wedge \text{NEXT}(x, x') \Rightarrow i(x') \end{cases}$$

Complétude et correction

$$\forall x_0, x \in \text{VALS}. \text{Init}(x_0) \wedge \text{NEXT}^*(x_0, x) \Rightarrow A(x_0, x).$$

si, et seulement si,

il existe $R \in \mathcal{P}(\text{VALS} \times \text{VALS})$

$$\forall x_0, x, x' \in \text{VALS} : \begin{cases} (1) \text{Init}(x_0) \Rightarrow R(x_0, x_0) \\ (2) R(x_0, x) \Rightarrow A(x_0, x) \\ (3) R(x_0, x) \wedge \text{NEXT}(x, x') \Rightarrow R(x_0, x') \end{cases}$$

si, et seulement si,

$\exists R \in \mathcal{P}(\text{VALS} \times \text{VALS}).$

$$\left[\begin{array}{l} (1) \text{Init} \times \text{Init} \subseteq R \\ (2) R \subseteq A \\ (3) \forall x, x' \in \text{VALS}. R(x_0, x) \wedge \text{NEXT}(x, x') \Rightarrow R(x_0, x') \end{array} \right.$$

- ▶ $\forall x_0, x \in \text{VALS}. \text{Init}(x_0) \wedge \text{NEXT}^*(x_0, x) \Rightarrow R(x_0, x)$ (R) est une propriété relationnelle de sûreté.
- ▶ Soit $y = (x_0, x)$, $y_0 = (x_0, x_0)$, et
$$\text{NEXTR}(y, y') \stackrel{\text{def}}{=} \text{NEXT}(x, x') \wedge y = (x_0, x) \wedge y' = (x_0, x')$$
- ▶ (R) est réécrit comme suit :
$$\forall y_0, y \in \text{VALS} \times \text{VALS}. \text{Init}(x_0) \wedge y_0 = (x_0, x_0) \wedge \text{NEXTR}^*(y_0, y) \Rightarrow R(y) \text{ (R)}$$
- ▶ Par la propriété de correction et de complétude
- ▶ il existe une propriété d'état $IR(y)$, telle que :

$$\forall y_0, y \in \text{VALS} \times \text{VALS}. \begin{cases} (1) \text{Init}(x_0) \wedge y_0 = (x_0, x_0) \Rightarrow IR(y) \\ (2) IR(y) \Rightarrow R(y) \\ (3) IR(y) \wedge \text{NEXTR}(y, y') \Rightarrow IR(y') \end{cases}$$

- ▶ il existe une propriété relationnelle $IR(x_0, x)$, telle que :

$$\forall x_0, x \in \text{VALS}. \begin{cases} (1) \text{Init}(x_0) \wedge \Rightarrow IR(x_0, x) \\ (2) IR(x_0, x) \Rightarrow R(x_0, x) \\ (3) IR(x_0, x) \wedge \text{NEXT}(x, x') \Rightarrow IR(x_0, x') \end{cases}$$

On obtient donc deux types de principes d'induction selon les propriétés de sûreté.

Complétude et correction

$\forall x_0, x \in \text{VALS}. \text{Init}(x_0) \wedge \text{NEXT}^*(x_0, x) \Rightarrow A(x).$

si, et seulement si,

il existe $I \in \mathcal{P}(\text{VALS})$

$$\forall x_0, x, x' \in \text{VALS} : \begin{cases} (1) \text{Init}(x_0) \Rightarrow I(x_0) \\ (2) I(x) \Rightarrow A(x) \\ (3) I(x) \wedge \text{NEXT}(x, x') \Rightarrow I(x') \end{cases}$$

- L'absence d'erreurs à l'exécution est caractérisée comme une propriété assertionnelle, puisqu'elle porte sur le fait qu'un état est sans erreurs à l'exécution si les calculs sont définis en cet état.
principe

Complétude et correction

$$\forall x_0, x \in \text{VALS}. \text{Init}(x_0) \wedge \text{NEXT}^*(x_0, x) \Rightarrow A(x_0, x).$$

si, et seulement si,

il existe $R \in \mathcal{P}(\text{VALS} \times \text{VALS})$

$$\forall x_0, x, x' \in \text{VALS} : \begin{cases} (1) \text{Init}(x_0) \Rightarrow R(x_0, x_0) \\ (2) R(x_0, x) \Rightarrow A(x_0, x) \\ (3) R(x_0, x) \wedge \text{NEXT}(x, x') \Rightarrow R(x_0, x') \end{cases}$$

- La correction partielle est caractérisée comme une relation entre l'état initial et l'état courant.

Principe assertionnel de sûreté ou d'invariance

$$\exists I(x) \in \mathcal{P}(\text{VALS}). \left[\begin{array}{l} \forall x, x' \in \text{VALS}. \\ (1) \text{ Init}(x) \Rightarrow I(x) \\ (2) I(x) \Rightarrow A(x) \\ (3) I(x) \wedge \text{NEXT}(x, x') \Rightarrow I(x') \end{array} \right.$$

Principe relationnel de sûreté ou d'invariance

$$\exists IR(x_0, x) \in \mathcal{P}(\text{VALS} \times \text{VALS}). \left[\begin{array}{l} \forall x_0, x, x' \in \text{VALS}. \\ (1) \text{ Init}(x_0) \Rightarrow IR(x_0, x_0) \\ (2) IR(x_0, x) \Rightarrow R(x_0, x) \\ (3) IR(x_0, x) \wedge \text{NEXT}(x, x') \Rightarrow IR(x_0, x') \end{array} \right.$$

① Le langage PlusCal

Checking a program or an algorithm using TLA^+ and PlusCal

Annotation of PlusCal programs

Define algorithms and processes in PlusCal

Macros et procédures

② Summation of the n first integers

③ Principe(s) d'induction

④ Méthode de preuves de propriétés d'invariance

- ▶ On considère un langage de programmation classique noté `PROGRAMS`
- ▶ et nous supposons que ce langage de programmation dispose de l'affectation, de la conditionnelle, de l'itération bornée, de l'itération non-bornée, de variables simples ou structurées comme les tableaux et de la définition de constantes.
- ▶ On se donne un programme `P` de `PROGRAMS` ; ce programme comprend
 - des variables notées globalement v ,
 - des constantes notées globalement pc ,
 - des types associés aux variables notés globalement `VALS` et identifiés à un ensemble de valeurs possibles des variables,
 - des instructions suivant un ordre défini par la syntaxe du langage de programmation.

- ▶ on définit un ensemble de points de contrôle `LOCATIONS`
- ▶ pour chaque programme ou algorithme `P`, `LOCATIONS` est un ensemble fini de valeurs et une variable cachée notée `pc` parcourt cet ensemble selon l'enchaînement.
- ▶ l'espace des valeurs possibles `VALS` est un produit cartésien de la forme `LOCATIONS × MEMORY`
- ▶ les variables x du système se décomposent en deux entités indépendantes $x = (pc, v)$ avec comme conditions $pc \in \text{LOCATIONS}$ et $v \in \text{MEMORY}$.

$$x = (pc, v) \wedge pc \in \text{LOCATIONS} \wedge v \in \text{MEMORY} \quad (5)$$

On considère un programme `P` annoté; on se donne un modèle relationnel

$\mathcal{MP} = (Th(s, c), x, \text{VALS}, \text{INIT}(x), \{r_0, \dots, r_n\})$ où

- ▶ $Th(s, c)$ est une théorie définissant les ensembles, les constantes et les propriétés statiques de ce programme
- ▶ x est une liste de variables flexibles et x comprend une partie contrôle et une partie mémoire.
- ▶ `LOCATIONS × MEMORY` est un ensemble de valeurs possibles pour x .
- ▶ $\{r_0, \dots, r_n\}$ est un ensemble fini de relations reliant les valeurs avant x et les valeurs après x' et conformes à la relation de succession $\longrightarrow$ entre les points de contrôle.
- ▶ $\text{INIT}(x)$ définit l'ensemble des valeurs initiales de (pc_0, v) et $x = (pc_0, v)$ avec $pre(v)$ qui caractérise les valeurs initiales de v au point initial.

On suppose qu'il existe un graphe sur l'ensemble des valeurs de contrôle définissant la relation de flux et nous notons cette structure $(\text{LOCATIONS}, \longrightarrow)$.

.....

☒ Definition

$$\ell_1 \longrightarrow \ell_2 \stackrel{\text{def}}{=} pc = \ell_1 \wedge pc' = \ell_2$$

.....

☒ Definition(Annotation d'un point de contrôle)

Soit une structure $(\text{LOCATIONS}, \longrightarrow)$ et une étiquette $\ell \in \text{LOCATIONS}$. Une annotation d'un point de contrôle ℓ est un prédicat $P_\ell(v)$ (version assertionnelle) ou $P_\ell(v_0, v)$ (version relationnelle).



$P_\ell(v_0, v)$ exprime une relation entre la valeur initiale de V notée v_0 et v la valeur courante de V au point ℓ et donc $P_\ell(v_0, v) \Rightarrow pre(v_0)$ précise que v_0 est une valeur initiale.

- ▶ Les étiquettes ℓ appartiennent à LOCATIONS : $\ell \in \text{LOCATIONS}$.
- ▶ Les variables v appartiennent à MEMORY : $v \in \text{MEMORY}$.
- ▶ $pre(v_0)$ spécifie les valeurs initiales de v .
- ▶ Chaque fois que le contrôle est en ℓ , v satisfait $P_\ell(v)$:
 $pc = \ell \Rightarrow P_\ell(v_0, v)$.
- ▶ A tout état (ℓ, v) du programme, la propriété suivante est vraie mais doit être prouvée :

$$J(\ell_0, v_0, pc, v) \stackrel{def}{=} \left[\begin{array}{l} \wedge pc \in \text{LOCATIONS} \\ \wedge v \in \text{MEMORY} \\ \dots \\ \wedge pc = \ell \Rightarrow P_\ell(v_0, v) \\ \dots \end{array} \right]$$

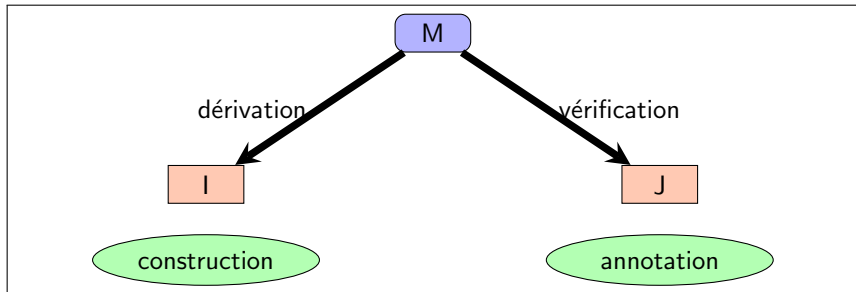
- ▶ $J(\ell_0, v_0, pc, v)$ est un invariant construit à partir des annotations produites mais il faut montrer que cet invariant permet de vérifier les trois conditions du principe d'induction.



ℓ_0 désigne l'étiquette marquant le début de l'algorithme et ℓ_f est la fin du programme. On pourra utiliser simplement 0 et f.

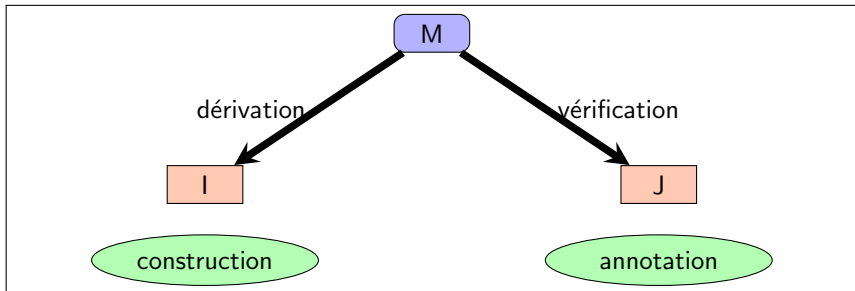
- ▶ Application de la correction du principe relationnel d'induction : si on vérifie les trois propriétés, alors A est une propriété de sûreté pour le modèle en question (vérification).
- ▶ Si on veut montrer que A est une propriété de sûreté, alors on doit utiliser l'invariant pour construire des annotations pour le modèle (dérivation).

- Si on veut montrer que A est une propriété de sûreté, alors on doit utiliser l'invariant pour construire des annotations pour le modèle (dérivation).



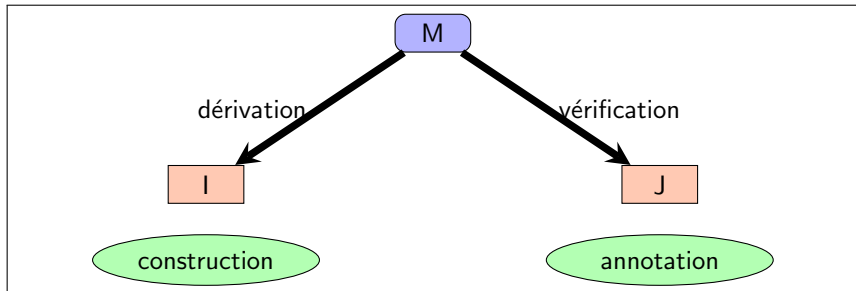
Utilisation du principe relationnel d'induction (RI)

- ▶ $\text{VALS} = \text{LOCATIONS} \times \text{MEMORY}$
- ▶ $J(pc_0, v_0, pc, v) \stackrel{def}{=} \exists x_0, x \in \text{VALS}. I(x_0, x) \wedge x = (pc, v) \wedge x_0 = (pc_0, v_0)$ (deduction)
- ▶ $I(x_0, x) \stackrel{def}{=} \exists pc_0, pc \in \text{LOCATIONS}, v_0, v \in \text{MEMORY}. J(pc_0, v_0, pc, v) \wedge x = (pc, v) \wedge x_0 = (pc_0, v_0)$ (induction)



Utilisation du principe assertionnel d'induction (AI)

- ▶ $\text{VALS} = \text{LOCATIONS} \times \text{MEMORY}$
- ▶ $J(pc, v) \stackrel{def}{=} \exists x \in \text{VALS}. I(x) \wedge x = (pc, v)$ (deduction)
- ▶ $I(x) \stackrel{def}{=} \exists pc \in \text{LOCATIONS}, v \in \text{MEMORY}. J(pc, v) \wedge x = (pc, v)$ (induction)



- (1) $\forall x_0, x \in \text{VALS}. \text{Init}(x_0) \wedge \text{NEXT}^*(x_0, x) \Rightarrow A(x) \ (I(x))$
- (2) $\forall x_0, x \in \text{VALS}. \text{Init}(x_0) \wedge \text{NEXT}^*(x_0, x) \Rightarrow R(x_0, x) \ (IR(x))$

Relations et définitions

$x = (\ell, v)$, $x_0 = (\ell_0, v_0)$, $I(x)$, $IR(x_0, x)$ et les annotations $P_\ell(v)$, $RP_\ell(v_0, v)$ sont liées ainsi :

- ▶ $I(x) \stackrel{\text{def}}{=} \exists \ell, v. (\ell \in \text{LOCATIONS} \wedge v \in \text{MEMORY} \wedge x = (\ell, v) \wedge P_\ell(v))$
- ▶ $P_\ell(v) \stackrel{\text{def}}{=} \exists x. (x \in \text{VALS} \wedge x = (\ell, v) \wedge I(x))$
- ▶ $IR(x_0, x) \stackrel{\text{def}}{=} \exists \ell, v, v_0. (\ell \in \text{LOCATIONS} \wedge v \in \text{MEMORY} \wedge x = (\ell, v) \wedge x_0 = (\ell_0, v_0) \wedge RP_\ell(v_0, v))$
- ▶ $RP_\ell(v_0, v) \stackrel{\text{def}}{=} \exists x, x_0. (x, x_0 \in \text{VALS} \wedge x = (\ell, v) \wedge x_0 = (\ell_0, v_0) \wedge IR(x_0, x))$

La transformation est fondée la relation de transition définie pour chaque couple d'étiquettes de contrôle qui se suivent est exprimée très simplement par la forme relationnelle suivante :

$$x \ r_{\ell, \ell'} \ x' \stackrel{\text{def}}{=} (pc = \ell \wedge \text{cond}_{\ell, \ell'}(v) \wedge v' = f_{\ell, \ell'}(v) \wedge pc' = \ell')$$

- ▶ La transition de ℓ à ℓ' est possible, quand la condition $cond_{\ell,\ell'}(v)$ est vraie pour V et quand le contrôle est en ℓ ($pc = \ell$).
- ▶ Quand la transition est observée, les variables V sont transformées comme suit $v' = f_{\ell,\ell'}(v)$.
- ▶ La définition de la transition n'exprime aucune hypothèse liée à une stratégie d'exécution comme l'équité par exemple.
- ▶ $cond_{\ell,\ell'}(v) \wedge v' = f_{\ell,\ell'}(v)$ est une expression où les expressions $cond_{\ell,\ell'}(v)$ et $v' = f_{\ell,\ell'}(v)$ posent des questions de définition :
 - $DOM(\ell, \ell')(v) \stackrel{def}{=} DEF(cond_{\ell,\ell'}(v))(v) \wedge DEF(f_{\ell,\ell'}(v))$
 - $DEF(E(X))(x)$, signifie que l'expression $E(X)$ est définie pour x la valeur courante de X .
- ▶ Certaines transitions peuvent conduire à des catastrophes :
 - $DEF(X+1)(x) \stackrel{def}{=} x+1 \in D$ où D est le domaine de codage de X par exemple $D = -2^{31} \dots 2^{31}-1$ pour un codage sur 32 bits.
 - $DEF(T(I+1) < V)(t, x, v) \stackrel{def}{=} i+1 \in dom(t) \wedge v \in D \wedge t(i+1) \in D$

Un programme P *remplit* un contrat $(pre, post)$:

- ▶ P transforme une variable v à partir d'une valeur initiale v_0 et produisant une valeur finale v_f : $v_0 \xrightarrow{P} v_f$
- ▶ v_0 satisfait pre : $pre(v_0)$ and v_f satisfait $post$: $post(v_0, v_f)$
- ▶ $pre(v_0) \wedge v_0 \xrightarrow{P} v_f \Rightarrow post(v_0, v_f)$

requires $pre(v_0)$

ensures $post(v_0, v_f)$

variables V

begin

$0 : P_0(v_0, v)$

instruction₀

...

$i : P_i(v_0, v)$

...

instruction _{$f-1$}

$f : P_f(v_0, v)$

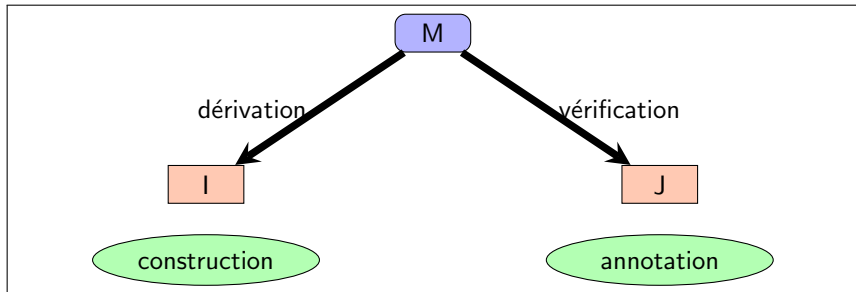
end

- ▶ $pre(v_0) \wedge v = v_0 \Rightarrow P_0(v_0, v)$
- ▶ $pre(v_0) \wedge P_f(v_0, v) \Rightarrow post(v_0, v)$
- ▶ conditions sur les transitions ℓ, ℓ' à définir à partir des principes d'induction.

```
variables  $U, V$   
requires  $u_0, v_0 \in \mathbb{N}$   
ensures  $u_f + v_f = u_0 + v_0$   
begin  
  0 :  $u = u_0 \wedge v = v_0 \wedge u_0, v_0 \in \mathbb{N}$   
   $U := U + 2$   
  1 :  $u = u_0 + 2 \wedge v = v_0 \wedge u_0, v_0 \in \mathbb{N}$   
   $V := V - 2$   
  2 :  $u = u_0 + 2 \wedge v = v_0 - 2 \wedge u_0, v_0 \in \mathbb{N}$   
end
```


- ▶ Application de la correction du principe relationnel d'induction : si on vérifie les trois propriétés, alors A est une propriété de sûreté pour le modèle en question (vérification).
- ▶ Si on veut montrer que A est une propriété de sûreté, alors on doit utiliser l'invariant pour construire des annotations pour le modèle (dérivation).

- Si on veut montrer que A est une propriété de sûreté, alors on doit utiliser l'invariant pour construire des annotations pour le modèle (dérivation).



► $x = (pc, u, v)$

► $J(0, u_0, v_0, pc, u, v) \stackrel{def}{=} \left[\begin{array}{l} \wedge pc \in \{0, 1, 2\} \\ \wedge u, v \in \mathbb{Z} \\ \wedge pc = 0 \Rightarrow u = u_0 \wedge v = v_0 \wedge u_0, v_0 \in \mathbb{N} \\ \wedge pc = 1 \Rightarrow u = u_0 + 2 \wedge v = v_0 \wedge u_0, v_0 \in \mathbb{N} \\ \wedge pc = 2 \Rightarrow u = u_0 + 2 \wedge v = v_0 - 2 \wedge u_0, v_0 \in \mathbb{N} \end{array} \right.$

► $A(0, u_0, v_0, pc, u, v) \stackrel{def}{=} (pc = 2 \Rightarrow u + v = u_0 + v_0 - 2 \wedge u_0, v_0 \in \mathbb{N})$

$\forall pc, u, v, pc', u', v' \in \{0, 1, 2\} \times \mathbb{Z} :$

$$\left\{ \begin{array}{l} (1) \text{ INIT}(0, u_0, v_0) \Rightarrow J(0, u_0, v_0, 0, u_0, v_0) \\ (2) J(0, u_0, v_0, pc, u, v) \Rightarrow A(0, u_0, v_0, pc, u, v) \\ (3) \forall i \in \{0, \dots, n\} : J(0, u_0, v_0, pc, u, v) \wedge x \text{ } r_i \text{ } pc', u', v' \Rightarrow J(0, u_0, v_0, pc', u', v') \end{array} \right.$$

ification 1 $[A \wedge (A \Rightarrow B)] \longrightarrow [A \wedge B]$

ification 2 $[A \wedge (B = C) \wedge D \Rightarrow E \wedge (B = F) \wedge G] \longrightarrow [A \wedge (B = C) \wedge D \Rightarrow E \wedge (C = F) \wedge G]$

ification 3 $[A \wedge (B = C) \wedge D \Rightarrow E \wedge (F = F) \wedge G] \longrightarrow [A \wedge (B = C) \wedge D \Rightarrow E \wedge TRUE \wedge G]$

ification 4 $[A \Rightarrow B \wedge TRUE \wedge C] \longrightarrow [A \Rightarrow B \wedge C]$



ification 5 $[A \wedge (B = C \Rightarrow U) \wedge (B = D \wedge B = C \Rightarrow V) \wedge C \neq D \wedge E] \longrightarrow [A \wedge B = C \wedge U \wedge C \neq D \wedge E]$

Le modèle relationnel $M(P)$ pour le programme P annoté est donc défini comme suit :

$$M(P) \stackrel{def}{=} (Th(s, c), (pc, v), \text{LOCATIONS} \times \text{MEMORY}, \text{Init}(\ell, v), \{r_{\ell, \ell'} \mid \ell, \ell' \in \text{LOCATIONS} \wedge \ell \longrightarrow \ell'\}).$$

La définition de $\text{Init}(x)$ est dépendante de la précondition de P :

$$\text{Init}(x) \stackrel{def}{=} .x = (\ell_0, v) \wedge \mathbf{pre}(P)(v).$$

Conditions initiales

Les deux propriétés suivantes sont équivalentes :

- ▶ $\forall x_0 \in \text{VALS} : \text{Init}(x_0) \Rightarrow J(x_0, x_0)$
- ▶ $\forall v \in \text{MEMORY}. \mathbf{pre}(P)(v) \wedge v = v_0 \Rightarrow P_{\ell_0}(v_0, v)$

- ▶ Les relations r_i correspondent aux transitions satisfaisant $\ell \longrightarrow \ell'$ et on associe à chaque r_i la relation $r_{\ell,\ell'}$
 - ▶ $x \ r_{\ell,\ell'} \ x' \stackrel{def}{=} (pc = \ell \wedge cond_{\ell,\ell'}(v) \wedge \wedge v' = f_{\ell,\ell'}(v) \wedge pc' = \ell')$
- ▶ $J(x_0, x) \stackrel{def}{=} \exists v_0, \ell, v. (\ell \in \text{LOCATIONS} \wedge v_0, v \in \text{MEMORY} \wedge x = (\ell, v) \wedge P_\ell(v_0, v))$
- ▶ $P_\ell(v_0, v) \stackrel{def}{=} \exists x_0, x. (x_0, x \in \text{VALS} \wedge x = (\ell, v) \wedge x_0 = (\ell_0, v_0) \wedge J(x_0, x))$

Pas d'induction

Les deux propriétés suivantes sont équivalentes :

- ▶ $\forall i \in \{0, \dots, n\} : J(x_0, x) \wedge x \ r_i \ x' \Rightarrow J(x_0, x')$
- ▶ $\forall \ell, \ell' \in \text{LOCATIONS} : \ell \longrightarrow \ell' \Rightarrow P_\ell(v_0, v) \wedge cond_{\ell,\ell'}(v) \wedge v' = f_{\ell,\ell'}(v) \Rightarrow P_{\ell'}(v_0, v')$

► $J(x_0 n x) \wedge x \ r_{\ell, \ell'} \ x' \Rightarrow J(x_0, x')$

- ▶ $J(x_0 n x) \wedge x \ r_{\ell, \ell'} \ x' \Rightarrow J(x_0, x')$
- ▶ $x \ r_{\ell, \ell'} \ x' \stackrel{def}{=} (pc = \ell \wedge cond_{\ell, \ell'}(v) \wedge \wedge v' = f_{\ell, \ell'}(v) \wedge pc' = \ell')$

- ▶ $J(x_0 n x) \wedge x \ r_{\ell, \ell'} \ x' \Rightarrow J(x_0, x')$
- ▶ $x \ r_{\ell, \ell'} \ x' \stackrel{def}{=} (pc = \ell \wedge cond_{\ell, \ell'}(v) \wedge \wedge v' = f_{\ell, \ell'}(v) \wedge pc' = \ell')$
- ▶ $I(x) \stackrel{def}{=} \exists \ell, v. (\ell \in \text{LOCATIONS} \wedge v \in \text{MEMORY} \wedge x = (\ell, v) \wedge P_\ell(v))$

- ▶ $J(x_0, x) \wedge x \text{ r}_{\ell, \ell'} x' \Rightarrow J(x_0, x')$
- ▶ $x \text{ r}_{\ell, \ell'} x' \stackrel{def}{=} (pc = \ell \wedge cond_{\ell, \ell'}(v) \wedge \wedge v' = f_{\ell, \ell'}(v) \wedge pc' = \ell')$
- ▶ $I(x) \stackrel{def}{=} \exists \ell, v. (\ell \in \text{LOCATIONS} \wedge v \in \text{MEMORY} \wedge x = (\ell, v) \wedge P_\ell(v))$
- ▶ $P_\ell(v_0, v) \stackrel{def}{=} \exists x. (x \in \text{VALS} \wedge x = (\ell, v) \wedge x_0 = (\ell_0, v_0) \wedge J(x_0, x))$

- ▶ $J(x_0 n x) \wedge x \ r_{\ell, \ell'} \ x' \Rightarrow J(x_0, x')$
- ▶ $x \ r_{\ell, \ell'} \ x' \stackrel{def}{=} (pc = \ell \wedge cond_{\ell, \ell'}(v) \wedge \wedge v' = f_{\ell, \ell'}(v) \wedge pc' = \ell')$
- ▶ $I(x) \stackrel{def}{=} \exists \ell, v. (\ell \in \text{LOCATIONS} \wedge v \in \text{MEMORY} \wedge x = (\ell, v) \wedge P_{\ell}(v))$
- ▶ $P_{\ell}(v_0, v) \stackrel{def}{=} \exists x. (x \in \text{VALS} \wedge x = (\ell, v) \wedge x_0 = (\ell_0, v_0) \wedge J(x_0, x))$
- ▶ $J(x_0 n x) \equiv pc = \ell \wedge P_{\ell}(v_0, v)$

- ▶ $J(x_0nx) \wedge x \ r_{\ell,\ell'} \ x' \Rightarrow J(x_0, x')$
- ▶ $x \ r_{\ell,\ell'} \ x' \stackrel{def}{=} (pc = \ell \wedge cond_{\ell,\ell'}(v) \wedge \wedge v' = f_{\ell,\ell'}(v) \wedge pc' = \ell')$
- ▶ $I(x) \stackrel{def}{=} \exists \ell, v. (\ell \in \text{LOCATIONS} \wedge v \in \text{MEMORY} \wedge x = (\ell, v) \wedge P_\ell(v))$
- ▶ $P_\ell(v_0, v) \stackrel{def}{=} \exists x. (x \in \text{VALS} \wedge x = (\ell, v) \wedge x_0 = (\ell_0, v_0) \wedge J(x_0, x))$
- ▶ $J(x_0nx) \equiv pc = \ell \wedge P_\ell(v_0, v)$
- ▶ $J(x_0, x') \equiv pc = \ell' \wedge P_{\ell'}(v_0, v')$

- ▶ $J(x_0nx) \wedge x \ r_{\ell,\ell'} \ x' \Rightarrow J(x_0, x')$
- ▶ $x \ r_{\ell,\ell'} \ x' \stackrel{def}{=} (pc = \ell \wedge cond_{\ell,\ell'}(v) \wedge \wedge v' = f_{\ell,\ell'}(v) \wedge pc' = \ell')$
- ▶ $I(x) \stackrel{def}{=} \exists \ell, v. (\ell \in \text{LOCATIONS} \wedge v \in \text{MEMORY} \wedge x = (\ell, v) \wedge P_\ell(v))$
- ▶ $P_\ell(v_0, v) \stackrel{def}{=} \exists x. (x \in \text{VALS} \wedge x = (\ell, v) \wedge x_0 = (\ell_0, v_0) \wedge J(x_0, x))$
- ▶ $J(x_0nx) \equiv pc = \ell \wedge P_\ell(v_0, v)$
- ▶ $J(x_0, x') \equiv pc = \ell' \wedge P_{\ell'}(v_0, v')$
- ▶ $pc = \ell \wedge P_\ell(v_0, v) \wedge (pc = \ell \wedge cond_{\ell,\ell'}(v) \wedge \wedge v' = f_{\ell,\ell'}(v) \wedge pc' = \ell' \Rightarrow pc = \ell' \wedge P_{\ell'}(v_0, v'))$

- $J(x_0 n x) \wedge x \text{ r}_{\ell, \ell'} x' \Rightarrow J(x_0, x')$
- $x \text{ r}_{\ell, \ell'} x' \stackrel{\text{def}}{=} (pc = \ell \wedge \text{cond}_{\ell, \ell'}(v) \wedge \wedge v' = f_{\ell, \ell'}(v) \wedge pc' = \ell')$
- $I(x) \stackrel{\text{def}}{=} \exists \ell, v. (\ell \in \text{LOCATIONS} \wedge v \in \text{MEMORY} \wedge x = (\ell, v) \wedge P_\ell(v))$
- $P_\ell(v_0, v) \stackrel{\text{def}}{=} \exists x. (x \in \text{VALS} \wedge x = (\ell, v) \wedge x_0 = (\ell_0, v_0) \wedge J(x_0, x))$
- $J(x_0 n x) \equiv pc = \ell \wedge P_\ell(v_0, v)$
- $J(x_0, x') \equiv pc = \ell' \wedge P_{\ell'}(v_0, v')$
- $pc = \ell \wedge P_\ell(v_0, v) \wedge (pc = \ell \wedge \text{cond}_{\ell, \ell'}(v) \wedge \wedge v' = f_{\ell, \ell'}(v) \wedge pc' = \ell' \Rightarrow pc = \ell' \wedge P_{\ell'}(v_0, v'))$
- $pc = \ell \wedge P_\ell(v_0, v) \wedge (pc = \ell \wedge \text{cond}_{\ell, \ell'}(v) \wedge \wedge v' = f_{\ell, \ell'}(v) \wedge pc' = \ell' \Rightarrow pc = \ell' \text{ (Tautologie)})$
- $pc = \ell \wedge P_\ell(v) \wedge (pc = \ell \wedge \text{cond}_{\ell, \ell'}(v) \wedge \wedge v' = f_{\ell, \ell'}(v) \wedge pc' = \ell' \Rightarrow P_{\ell'}(v_0, v'))$

- $J(x_0nx) \wedge x \ r_{\ell,\ell'} \ x' \Rightarrow J(x_0, x')$
- $x \ r_{\ell,\ell'} \ x' \stackrel{def}{=} (pc = \ell \wedge cond_{\ell,\ell'}(v) \wedge \wedge v' = f_{\ell,\ell'}(v) \wedge pc' = \ell')$
- $I(x) \stackrel{def}{=} \exists \ell, v. (\ell \in \text{LOCATIONS} \wedge v \in \text{MEMORY} \wedge x = (\ell, v) \wedge P_\ell(v))$
- $P_\ell(v_0, v) \stackrel{def}{=} \exists x. (x \in \text{VALS} \wedge x = (\ell, v) \wedge x_0 = (\ell_0, v_0) \wedge J(x_0, x))$
- $J(x_0nx) \equiv pc = \ell \wedge P_\ell(v_0, v)$
- $J(x_0, x') \equiv pc = \ell' \wedge P_{\ell'}(v_0, v')$
- $pc = \ell \wedge P_\ell(v_0, v) \wedge (pc = \ell \wedge cond_{\ell,\ell'}(v) \wedge \wedge v' = f_{\ell,\ell'}(v) \wedge pc' = \ell' \Rightarrow pc = \ell' \wedge P_{\ell'}(v_0, v'))$
- $pc = \ell \wedge P_\ell(v_0, v) \wedge (pc = \ell \wedge cond_{\ell,\ell'}(v) \wedge \wedge v' = f_{\ell,\ell'}(v) \wedge pc' = \ell' \Rightarrow pc = \ell' \text{ (Tautologie)})$
- $pc = \ell \wedge P_\ell(v) \wedge (pc = \ell \wedge cond_{\ell,\ell'}(v) \wedge \wedge v' = f_{\ell,\ell'}(v) \wedge pc' = \ell' \Rightarrow P_{\ell'}(v_0, v'))$
- $P_\ell(v_0, v) \wedge cond_{\ell,\ell'}(v) \wedge v' = f_{\ell,\ell'}(v) \Rightarrow P_{\ell'}(v_0, v')$

- ## Conclusion

- ▶ $J(x_0, x) \Rightarrow A(x_0, x)$
- ▶ $\forall \ell \in \text{LOCATIONS}, v, v_0 \in \text{MEMORY}. P_\ell(v_0, v) \Rightarrow A(\ell_0, v_0, \ell, v)$

- ▶ Le programme est annoté.
- ▶ Les annotations définissent un invariant à vérifier selon les conditions de vérification.
- ▶ $A(\ell, v)$ est l'énoncé de la propriété de sûreté à vérifier.

Méthode relationnelle de correction de propriétés de sûreté

Soit $A(\ell_0, v_0, \ell, v)$ une propriété d'un programme P . Soit une famille d'annotations famille de propriétés $\{P_\ell(v_0, v) : \ell \in \text{LOCATIONS}\}$ pour ce programme. Si les conditions suivantes sont vérifiées :
alors $A(\ell_0, v_0, \ell, v)$ est une propriété de sûreté pour le programme P .

☒ **Definition** Condition de vérification

L'expression $P_\ell(v_0, v) \wedge cond_{\ell, \ell'}(v) \wedge v' = f_{\ell, \ell'}(v) \Rightarrow P_{\ell'}(v_0, v')$ où ℓ, ℓ' sont deux étiquettes liées par la relation $\longrightarrow$, est appelée une condition de vérification.

Floyd and Hoare

- $\forall v_0, v, v' \in \text{MEMORY}. \forall \ell, \ell' \in \text{LOCATIONS}. \ell \longrightarrow \ell' \Rightarrow$
 $P_\ell(v_0, v) \wedge \text{cond}_{\ell, \ell'}(v) \wedge v' = f_{\ell, \ell'}(v) \Rightarrow P_{\ell'}(v_0, v')$ est équivalent à
 $\forall \ell, \ell' \in \text{LOCATIONS}. \ell \longrightarrow \ell' \Rightarrow \forall v' \in$
 $\text{MEMORY}. P_\ell(v_0, v) \wedge \text{cond}_{\ell, \ell'}(v) \Rightarrow P_{\ell'}(v_0, v \mapsto f_{\ell, \ell'}(v))$
- $\forall v_0, v, v' \in \text{MEMORY}. \forall \ell, \ell' \in \text{LOCATIONS}. \ell \longrightarrow \ell' \Rightarrow$
 $P_\ell(v_0, v) \wedge \text{cond}_{\ell, \ell'}(v) \wedge v' = f_{\ell, \ell'}(v) \Rightarrow P_{\ell'}(v_0, v')$ est équivalent à
 $\forall \ell, \ell' \in \text{LOCATIONS}. \ell \longrightarrow \ell' \Rightarrow \forall v' \in$
 $\text{MEMORY}. (\exists v \in \text{MEMORY}. P_\ell(v_0, v) \wedge \text{cond}_{\ell, \ell'}(v) \wedge v' = f_{\ell, \ell'}(v)) \Rightarrow$
 $P_{\ell'}(v_0, v')$

Nous pouvons resumer les deux formes possibles de l'affectation suivante :

$$\blacktriangleright \forall v, v' \in \text{MEMORY}. P_\ell(v_0, v) \wedge \text{cond}_{\ell, \ell'}(v) \wedge v' = f_{\ell, \ell'}(v) \Rightarrow P_{\ell'}(v_0, v')$$

$$\begin{array}{l} \ell : P_\ell(v_0, v) \\ V := f_{\ell, \ell'}(V) \\ \ell' : P_{\ell'}(v_0, v) \end{array}$$

Nous pouvons resumer les deux formes possibles de l'affectation suivante :

- ▶ $\forall v, v' \in \text{MEMORY}. P_\ell(v_0, v) \wedge \text{cond}_{\ell, \ell'}(v) \wedge v' = f_{\ell, \ell'}(v) \Rightarrow P_{\ell'}(v_0, v')$
- ▶ $\forall v, v' \in \text{MEMORY}. P_\ell(v_0, v) \wedge \text{TRUE} \wedge v' = f_{\ell, \ell'}(v) \Rightarrow P_{\ell'}(v_0, v')$

$$\begin{aligned} \ell &: P_\ell(v_0, v) \\ V &:= f_{\ell, \ell'}(V) \\ \ell' &: P_{\ell'}(v_0, v) \end{aligned}$$

Nous pouvons resumer les deux formes possibles de l'affectation suivante :

- ▶ $\forall v, v' \in \text{MEMORY}. P_\ell(v_0, v) \wedge \text{cond}_{\ell, \ell'}(v) \wedge v' = f_{\ell, \ell'}(v) \Rightarrow P_{\ell'}(v_0, v')$
- ▶ $\forall v, v' \in \text{MEMORY}. P_\ell(v_0, v) \wedge \text{TRUE} \wedge v' = f_{\ell, \ell'}(v) \Rightarrow P_{\ell'}(v_0, v')$
- ▶ $\forall v, v' \in \text{MEMORY}. P_\ell(v_0, v) \wedge v' = f_{\ell, \ell'}(v) \Rightarrow P_{\ell'}(v_0, v')$

$$\begin{aligned} \ell &: P_\ell(v_0, v) \\ V &:= f_{\ell, \ell'}(V) \\ \ell' &: P_{\ell'}(v_0, v) \end{aligned}$$

Nous pouvons resumer les deux formes possibles de l'affectation suivante :

$$\begin{aligned}\ell &: P_\ell(v_0, v) \\ V &:= f_{\ell, \ell'}(V) \\ \ell' &: P_{\ell'}(v_0, v)\end{aligned}$$

- ▶ $\forall v, v' \in \text{MEMORY}. P_\ell(v_0, v) \wedge \text{cond}_{\ell, \ell'}(v) \wedge v' = f_{\ell, \ell'}(v) \Rightarrow P_{\ell'}(v_0, v')$
- ▶ $\forall v, v' \in \text{MEMORY}. P_\ell(v_0, v) \wedge \text{TRUE} \wedge v' = f_{\ell, \ell'}(v) \Rightarrow P_{\ell'}(v_0, v')$
- ▶ $\forall v, v' \in \text{MEMORY}. P_\ell(v_0, v) \wedge v' = f_{\ell, \ell'}(v) \Rightarrow P_{\ell'}(v_0, v')$
- ▶ $\forall v \in \text{MEMORY}. P_\ell(v_0, v) \Rightarrow P_{\ell'}(v_0, v \mapsto f_{\ell, \ell'}(v))$
(l'axiomatique de Hoare).

Nous pouvons resumer les deux formes possibles de l'affectation suivante :

$$\begin{array}{l} \ell : P_\ell(v_0, v) \\ V := f_{\ell, \ell'}(V) \\ \ell' : P_{\ell'}(v_0, v) \end{array}$$

- ▶ $\forall v, v' \in \text{MEMORY}. P_\ell(v_0, v) \wedge \text{cond}_{\ell, \ell'}(v) \wedge v' = f_{\ell, \ell'}(v) \Rightarrow P_{\ell'}(v_0, v')$
- ▶ $\forall v, v' \in \text{MEMORY}. P_\ell(v_0, v) \wedge \text{TRUE} \wedge v' = f_{\ell, \ell'}(v) \Rightarrow P_{\ell'}(v_0, v')$
- ▶ $\forall v, v' \in \text{MEMORY}. P_\ell(v_0, v) \wedge v' = f_{\ell, \ell'}(v) \Rightarrow P_{\ell'}(v_0, v')$
- ▶ $\forall v \in \text{MEMORY}. P_\ell(v_0, v) \Rightarrow P_{\ell'}(v_0, v \mapsto f_{\ell, \ell'}(v))$
(l'axiomatique de Hoare).
- ▶ $\forall v \in \text{MEMORY}. (\exists v' \in \text{MEMORY}. P_\ell(v_0, v) \wedge v' = f_{\ell, \ell'}(v)) \Rightarrow P_{\ell'}(v_0, v')$
correspond à la règle d'affectation de Floyd.

```
 $\ell_1 : P_{\ell_1}(v_0, v)$   
WHILE  $B(v)$  DO  
   $\ell_2 : P_{\ell_2}(v_0, v)$   
  ...  
   $\ell_3 : P_{\ell_3}(v_0, v)$   
END  
 $\ell_4 : P_{\ell_4}(v_0, v)$ 
```

Pour la structure d'itération, les conditions de vérification sont les suivantes :

- ▶ $P_{\ell_1}(v_0, v) \wedge B(v) \Rightarrow P_{\ell_2}(v_0, v)$
- ▶ $P_{\ell_1}(v_0, v) \wedge \neg B(v) \Rightarrow P_{\ell_4}(v_0, v)$
- ▶ $P_{\ell_3}(v_0, v) \wedge B(v) \Rightarrow P_{\ell_2}(v_0, v)$
- ▶ $P_{\ell_3}(v_0, v) \wedge \neg B(v) \Rightarrow P_{\ell_4}(v_0, v)$

```
 $\ell_1 : P_{\ell_1}(v_0, v)$   
IF  $B(v)$  THEN  
   $\ell_2 : P_{\ell_2}(v_0, v)$   
  ...  
   $\ell_3 : P_{\ell_3}(v_0, v)$   
ELSE  
   $m_2 : P_{\ell_2}(v_0, v)$   
  ...  
   $m_3 : P_{\ell_3}(v_0, v)$   
FI  
 $\ell_4 : P_{\ell_4}(v_0, v)$ 
```

Pour la structure de conditionnelle, les conditions suivantes :

- ▶ $P_{\ell_1}(v_0, v) \wedge B(v) \Rightarrow P_{\ell_2}(v_0, v)$
- ▶ $P_{\ell_3}(v_0, v) \Rightarrow P_{\ell_4}(v_0, v)$
- ▶ $P_{\ell_1}(v_0, v) \wedge \neg B(v) \Rightarrow P_{m_2}(v_0, v)$
- ▶ $P_{m_3}(v_0, v) \Rightarrow P_{\ell_4}(v_0, v)$

Soit v une variable d'état de P . **pre**(P)(v) est la précondition de P pour v ; elle caractérise les valeurs initiales de v . **post**(P)(v_0, v) est la postcondition de P pour v ; elle caractérise les valeurs finales de v en relation avec la valeur initiale v_0

Exemple

- 1 **pre**(P)(x, y, z)= $x, y, z \in \mathbb{N}$ et **post**(P)(x_0, y_0, z_0, x, y, z)= $z = x_0 \cdot y_0$
- 2 **pre**(Q)(x, y, z)= $x, y, z \in \mathbb{N}$ et
post(Q)(x_0, y_0, z_0, x, y, z)= $z = x_0 + y_0$

$$\forall \underline{x}, \underline{y}, \underline{r}, \underline{q}, \bar{x}, \bar{y}, \bar{r}, \bar{q}.$$

$$\mathbf{pre}(P)(\underline{x}, \underline{y}, \underline{r}, \underline{q}) \wedge (\underline{x}, \underline{y}, \underline{r}, \underline{q}) \xrightarrow{P} (\bar{x}, \bar{y}, \bar{r}, \bar{q}) \\ \Rightarrow \mathbf{post}(P)(\underline{x}, \underline{y}, \underline{r}, \underline{q}, \bar{x}, \bar{y}, \bar{r}, \bar{q})$$

La correction partielle vise à établir qu'un programme P est partiellement correct par rapport à sa précondition et à sa postcondition.

- ▶ la spécification des données de P **pre**(P)(v_0)
- ▶ la spécification des résultats de P **post**(P)(v_0, v)
- ▶ une famille d'annotations de propriétés $\{P_\ell(v_0, v) : \ell \in \text{LOCATIONS}\}$ pour ce programme.
- ▶ une propriété de sûreté définissant la correction partielle $pc = \ell_f \Rightarrow \mathbf{post}(P)(v_0, v_f)$ où ℓ_f est l'étiquette marquant la fin du programme P

.....

☒ Definition

Le programme P est partiellement correct par rapport à **pre**(P)(v_0) et **post**(P)(v_0, v), si la propriété $pc = \ell_f \Rightarrow \mathbf{post}(P)(v_0, v)$ est une propriété de sûreté pour ce programme.

.....

Si les conditions suivantes sont vérifiées :

- ▶ $\forall v_0, v \in \text{MEMORY} : \mathbf{pre}(P)(v_0) \wedge v = v_0 \Rightarrow P_{\ell_0}(v_0, v)$
- ▶ $\forall v_0, v \in \text{MEMORY} : P_{\ell_f}(v_0, v) \Rightarrow \mathbf{post}(P)(v_0, v)$
- ▶ $\forall \ell, \ell' \in \text{LOCATIONS} : \ell \longrightarrow \ell' : \forall v_0, v, v' \in \text{MEMORY}. (P_{\ell}(v_0, v) \wedge \text{cond}_{\ell, \ell'}(v) \wedge v' = f_{\ell, \ell'}(v) \Rightarrow P_{\ell'}(v_0, v'))$,

alors le programme P est partiellement correct par rapport à $\mathbf{pre}(P)(v_0)$ et $\mathbf{post}(P)(v_0, v)$.

- ▶ La correction partielle indique que si le programme termine normalement, alors la postcondition est vérifiée par les variables courantes.
- ▶ La sémantique du contrat est donc assez simple à donner :

- $$\text{PC}(x_0x) \stackrel{def}{=} x_0 = (\ell_0, v_0) \wedge x = (pc, v) \Rightarrow (pc = \ell_f \Rightarrow \text{post}(v_0, v_f))$$



An Early Program Proof by Alan Turing

Turing, A. M. 1949. "Checking a Large Routine." In Report of a Conference on High Speed Automatic Calculating Machines, Univ. Math. Lab., Cambridge, pp. 67-69.

- ▶ Turing se pose une question fondamentale de la correction des routines ou programmes en 1949.
- ▶ Il s'agit sans doute (Jones!) de la méthode d'annotation et d'induction sur les programmes qui sera finalisée par Floyd en 1967.

Méthode de Floyd

- ▶ Au point 0, $pre(x_0) \wedge x = x_0 \Rightarrow P_0(x_0, x)$
- ▶ Annotations : au point i , l'assertion $P_i(x_0, x)$ est vraie.
- ▶ Au point final f , $pre(x_0) \wedge P_f(x_0, x) \Rightarrow post(x_0, x)$

- ▶ La transition à exécuter est celle allant de ℓ à ℓ' et caractérisée par la condition ou garde $cond_{\ell,\ell'}(v)$ sur v et une transformation de la variable v , $v' = f_{\ell,\ell'}(v)$.
- ▶ Une condition d'absence d'erreur est définie par $\mathbf{DOM}(\ell, \ell')(v)$ pour la transition considérée. $\mathbf{DOM}(\ell, \ell')(v)$ signifie que la transition $\ell \longrightarrow \ell'$ est possible et ne conduit pas à une erreur.
- ▶ Une erreur est un débordement arithmétique, une référence à un élément de tableau qui n'existe pas, une référence à un pointeur nul, ...

exemple

- 1 La transition correspond à une affectation de la forme $x := x+y$ ou $y := x+y$:
$$\mathbf{DOM}(x+y)(x, y) \stackrel{def}{=} \mathbf{DOM}(x)(x, y) \wedge \mathbf{DOM}(y)(x, y) \wedge x+y \in int$$
- 2 La transition correspond à une affectation de la forme $x := x+1$ ou $y := x+1$:
$$\mathbf{DOM}(x+1)(x, y) \stackrel{def}{=} \mathbf{DOM}(x)(x, y) \wedge x+2 \in int$$

Définition RTE

L'absence d'erreurs à l'exécution vise à établir qu'un programme P ne va pas produire des erreurs durant son exécution par rapport à sa précondition et à sa postcondition.

- ▶ la spécification des données de P **pre**(P)(v)
- ▶ la spécification des résultats de P **post**(P)(v_0, v)
- ▶ une famille d'annotations de propriétés $\{P_\ell(v) : \ell \in \text{LOCATIONS}\}$ pour ce programme.
- ▶ une propriété de sûreté définissant l'absence d'erreurs à l'exécution :

$$\bigwedge_{\ell \in \text{LOCATIONS} - \{\text{output}\}, n \in \text{LOCATIONS}, \ell \longrightarrow n} (\mathbf{DOM}(\ell, n)(v))$$

.....

☒ Definition

Le programme P ne produira pas d'erreurs à l'exécution par rapport à **pre**(P)(v) et **post**(P)(v_0, v), si la propriété

$$\bigwedge_{\ell \in \text{LOCATIONS} - \{\text{output}\}, n \in \text{LOCATIONS}, \ell \longrightarrow n} (\mathbf{DOM}(\ell, n)(v))$$
 est une propriété de sûreté pour ce programme.

RTE = Run Time Error

Si les conditions suivantes sont vérifiées :

- ▶ $\forall v_0, v \in \text{MEMORY} : \mathbf{pre}(P)(v_0) \wedge v = v_0 \Rightarrow P_{\ell_0}(v_0, v)$
- ▶ $\forall m \in \text{LOCATIONS} - \{\ell_f\}, n \in \text{LOCATIONS}, \forall v_0, v, v' \in \text{MEMORY} : m \longrightarrow n : \mathbf{pre}(P)(v_0) \wedge P_m(v_0, v) \Rightarrow \mathbf{DOM}(m, n)(v)$
- ▶ $\forall \ell, \ell' \in \text{LOCATIONS} : \ell \longrightarrow \ell' : \forall v_0, v, v' \in \text{MEMORY}. (P_\ell(v_0, v) \wedge \text{cond}_{\ell, \ell'}(v) \wedge v' = f_{\ell, \ell'}(v) \Rightarrow P_{\ell'}(v_0, v'))$,

alors le programme P ne produira pas d'erreurs à l'exécution par rapport à $\mathbf{pre}(P)(v_0)$ et $\mathbf{post}(P)(v_0, v)$.

- ▶ On doit d'abord vérifier la correction partielle puis renforcer les assertions de la correction partielle par des conditions de domaine.
- ▶ On peut donc en déduire un contrat qui intègre aussi la vérification de l'absence d'erreurs à l'exécution.

Un programme P *remplit* un contrat (pre,post) :

- ▶ P transforme une variable v à partir d'une valeur initiale v_0 et produisant une valeur finale v_f : $v_0 \xrightarrow{P} v_f$
- ▶ v_0 satisfait pre : $\text{pre}(v_0)$ and v_f satisfait post : $\text{post}(v_0, v_f)$
- ▶ $\text{pre}(v_0) \wedge v_0 \xrightarrow{P} v_f \Rightarrow \text{post}(v_0, v_f)$
- ▶ $\mathbb{D}$ est le domaine RTE de V

requires $\text{pre}(v_0)$
 ensures $\text{post}(v_0, v_f)$
 variables V

```
begin
  0 :  $P_0(v_0, v)$ 
  instruction0
  ...
  i :  $P_i(v_0, v)$ 
  ...
  instructionf-1
  f :  $P_f(v_0, v)$ 
end
```

- ▶ $\text{pre}(v_0) \wedge v = v_0 \Rightarrow P_0(v_0, v)$
- ▶ $\text{pre}(x_0) \wedge P_f(v_0, v) \Rightarrow \text{post}(v_0, v)$
- ▶ Pour toute paire d'étiquettes ℓ, ℓ' telle que $\ell \longrightarrow \ell'$, on vérifie que, pour toutes valeurs $v, v' \in \text{MEMORY}$

$$\left(\begin{array}{c} P_\ell(v_0, v) \\ \wedge \text{cond}_{\ell, \ell'}(v) \wedge v' = f_{\ell, \ell'}(v) \\ \Rightarrow P_{\ell'}(v_0, v') \end{array} \right),$$
- ▶ $\forall m \in \text{LOCATIONS} - \{\ell_f\}, n \in \text{LOCATIONS}, \forall v_0, v, v' \in \text{MEMORY} :$
 $m \longrightarrow n :$
 $\text{pre}(v_0) \wedge P_m(v_0, v) \Rightarrow \text{DOM}(m, n)(v)$